

CO1-AT3-Coding Challenge

View Only

[← Back](#)

QUESTIONS

Q1Smart University Payroll
Management System
25 marks**Q2**Smart Vehicle Insurance Claim
System
25 marks

2/2 answered

Q1 Smart University Payroll Management System

25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

Input

3

TeachingFaculty

101

Dr. Arun

CSE

50000

20

500

TechnicalStaff

102

Mr. Ravi

ECE

35000

15

300

VisitingFaculty

103

Dr. Kumar

IT

0

40

1200

Search Employee ID:

102

Expected Output

Employee Details

Employee ID : 101

Name : Dr. Arun

Department : CSE

Monthly Salary : 60000.0

Employee ID : 102
Name : Mr. Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Dr. Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee
Employee ID : 101
Salary : 60000.0

Average Salary
49166.67

Employee Found
Employee ID : 102
Name : Mr. Ravi
Salary : 39500.0

Your Code

```
1 import java.util.Scanner;  
2  
3 public class Main {  
4     public static void main(String[] args) {  
5         Scanner sc = new Scanner(System.in);  
6  
7         int n = Integer.parseInt(sc.nextLine().trim());
```

Input

3
TeachingFaculty
101
Dr.Arun

```

8      Employee[] emp = new Employee[n];
9
10     for (int i = 0; i < n; i++) {
11         String type = sc.nextLine().trim();
12         while (type.isEmpty())
13             type = sc.nextLine().trim();
14
15         int id = Integer.parseInt(sc.nextLine());
16         String name = sc.nextLine().trim();
17         String dept = sc.nextLine().trim();
18         double basicSalary = Double.parseDouble(sc.nextLine());
19         int value1 = Integer.parseInt(sc.nextLine());
20         double value2 = Double.parseDouble(sc.nextLine());
21
22         if (type.equalsIgnoreCase("TeachingFaculty"))
23             emp[i] = new TeachingFaculty(id, name, dept, basicSalary, value1, value2);
24         } else if (type.equalsIgnoreCase("TechnicalStaff"))
25             emp[i] = new TechnicalStaff(id, name, dept, basicSalary, value1, value2);
26         } else if (type.equalsIgnoreCase("VisitingFaculty"))
27             emp[i] = new VisitingFaculty(id, name, dept, basicSalary, value1, value2);
28         }
29     }
30
31     String temp = sc.nextLine().trim();
32     while (temp.isEmpty())
33         temp = sc.nextLine().trim();
34
35     int searchId;
36     if (temp.equalsIgnoreCase("Search Employee"))
37         searchId = Integer.parseInt(sc.nextLine());
38     else
39         searchId = Integer.parseInt(temp);
40

```

Output

Employee Details

Employee ID : 101

Name : Dr.Arun

Department : CSE

Monthly Salary :

```
41     System.out.println("Employee Details");
42     System.out.println();
43
44     Employee highest = emp[0];
45     double total = 0;
46
47     for (Employee e : emp) {
48         System.out.println("Employee ID : " + e.getId());
49         System.out.println("Name : " + e.getName());
50         System.out.println("Department : " + e.getDepartment());
51         System.out.println("Monthly Salary : " + e.getMonthlySalary());
52         System.out.println();
53
54         double sal = e.calculateSalary();
55         total += sal;
56
57         if (sal > highest.calculateSalary())
58             highest = e;
59     }
60
61     System.out.println("Highest Paid Employee");
62     System.out.println("Employee ID : " + highest.getId());
63     System.out.println("Salary : " + highest.calculateSalary());
64     System.out.println();
65
66     System.out.println("Average Salary");
67     System.out.printf("%.2f\n", (total / n));
68     System.out.println();
69
70     boolean found = false;
71     for (Employee e : emp) {
72         if (e.getEmployeeId() == searchId) {
73             found = true;
```

```
74         System.out.println("Employee Found")
75         System.out.println("Employee ID : " + e.getId())
76         System.out.println("Name : " + e.getName())
77         System.out.println("Salary : " + e.getSalary())
78         break;
79     }
80 }
81
82 if (!found) {
83     System.out.println("Employee Not Found")
84 }
85
86 sc.close();
87 }
88 }
89
90 class Employee {
91     private int employeeId;
92     private String employeeName;
93     private String department;
94     private double basicSalary;
95
96     Employee(int employeeId, String employeeName, String department, double basicSalary) {
97         this.employeeId = employeeId;
98         this.employeeName = employeeName;
99         this.department = department;
100        this.basicSalary = basicSalary;
101    }
102
103    public int getEmployeeId() {
104        return employeeId;
105    }
106}
```

```
107     public String getEmployeeName() {
108         return employeeName;
109     }
110
111     public String getDepartment() {
112         return department;
113     }
114
115     public double getBasicSalary() {
116         return basicSalary;
117     }
118
119     public double calculateSalary() {
120         return basicSalary;
121     }
122 }
123
124 class TeachingFaculty extends Employee {
125     int teachingHours;
126     double incentivePerHour;
127
128     TeachingFaculty(int id, String name, String dept,
129                     double salary, int h, double p) {
130         super(id, name, dept, salary);
131         teachingHours = h;
132         incentivePerHour = p;
133     }
134
135     @Override
136     public double calculateSalary() {
137         return getBasicSalary() + (teachingHours *
138                                     incentivePerHour);
139     }
```

```
140 class TechnicalStaff extends Employee {
141     int overtimeHours;
142     double overtimeRate;
143
144     TechnicalStaff(int id, String name, String dept,
145         super(id, name, dept, salary);
146         overtimeHours = h;
147         overtimeRate = r;
148     }
149
150     @Override
151     public double calculateSalary() {
152         return getBasicSalary() + (overtimeHours *
153     }
154 }
155
156 class VisitingFaculty extends Employee {
157     int lecturesHandled;
158     double paymentPerLecture;
159
160     VisitingFaculty(int id, String name, String dept,
161         super(id, name, dept, salary);
162         lecturesHandled = l;
163         paymentPerLecture = p;
164     }
165
166     @Override
167     public double calculateSalary() {
168         return lecturesHandled * paymentPerLecture;
169     }
170 }
```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.